

# Day 9 — Sun 13 Sep

**Countdown:** 13 days to General English & GK

**Method:** start by recalling yesterday's topic with this page closed. Study the sections below, work every example by hand, then do 20 MCQs.

## 🕒 GK — Polity II

Source: General/03\_GK\_CAPSULE.md

### 1.6 The Union Executive

**President (Art. 52–62):**

| Point         | Detail                                                                                                                                           |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Nature        | Constitutional (nominal) head; real power with Council of Ministers                                                                              |
| Election      | <b>Indirect</b> — by an electoral college of elected MPs + elected MLAs, by <b>single transferable vote</b> / <b>proportional representation</b> |
| Term          | 5 years; eligible for re-election                                                                                                                |
| Qualification | Citizen, 35+, qualified for Lok Sabha membership                                                                                                 |
| Removal       | <b>Impeachment</b> for "violation of the Constitution" (Art. 61)                                                                                 |
| Key powers    | Executive, legislative (assent to bills, ordinances Art. 123), judicial (pardon Art. 72), emergency, military                                    |

**Prime Minister & Council of Ministers (Art. 74–75):** The PM is appointed by the President (leader of the majority in Lok Sabha). Art. 74 — Council of Ministers headed by PM to **aid and advise** the President (advice is **binding**, 42nd/44th Amdt). The Council is **collectively responsible to the Lok Sabha** (Art. 75).

**Vice-President (Art. 63–71):** ex-officio Chairman of the **Rajya Sabha**; elected by both Houses (electoral college of MPs only).

### 1.7 Parliament

**Intuition.** Two Houses: Lok Sabha is "the people's house" (directly elected, powerful on money); Rajya Sabha is "the states' house" (permanent, represents states).

| Feature              | Lok Sabha (House of the People)                   | Rajya Sabha (Council of States)            |
|----------------------|---------------------------------------------------|--------------------------------------------|
| Max strength         | <b>552</b> (530 states + 20 UTs) △ verify current | <b>250</b> (238 elected + 12 nominated)    |
| Elected by           | People directly                                   | State legislatures (STV)                   |
| Term                 | 5 years (dissolvable)                             | <b>Permanent</b> — 1/3 retire every 2 yrs  |
| Presiding officer    | <b>Speaker</b>                                    | Vice-President (Chairman)                  |
| Money bills          | Originate <b>only here</b> (Art. 110)             | Can only recommend; must return in 14 days |
| No-confidence motion | Only here                                         | Cannot                                     |

**How a bill becomes law:** Introduction → First reading → Second reading (clause-by-clause) → Third reading → other House repeats → **President's assent** (Art. 111).

A **Money Bill** needs the President's prior recommendation and starts only in Lok Sabha. A deadlock on an *ordinary* bill is resolved by a **joint sitting (Art. 108)**, presided over by the **Speaker**.

## 1.8 The Judiciary

- **Supreme Court (Art. 124)**: integrated single judiciary; **judicial review** (can strike down laws violating the Constitution); guardian of Fundamental Rights.
- **Judges**: appointed by the President; retire at **65 (SC)** / **62 (HC)**.
- **Basic Structure Doctrine** — *Kesavananda Bharati v. State of Kerala (1973)*: Parliament can amend any part of the Constitution **but cannot alter its "basic structure"** (supremacy of the Constitution, secularism, separation of powers, federalism, judicial review, rule of law). A landmark, very high-yield.

## 1.13 Landmark amendments to memorise

| Amendment                         | Year | What it did                                                                                             |
|-----------------------------------|------|---------------------------------------------------------------------------------------------------------|
| <b>1st</b>                        | 1951 | Added the <b>9th Schedule</b> (protect land-reform laws from review)                                    |
| <b>42nd</b> ("Mini-Constitution") | 1976 | Added <i>Socialist, Secular, Integrity</i> to Preamble; Fundamental Duties; made DPSP superior in parts |
| <b>44th</b>                       | 1978 | Removed Right to Property from FRs (→ Art. 300A); "armed rebellion" in Art. 352; protected Art. 20 & 21 |
| <b>52nd</b>                       | 1985 | <b>Anti-defection law</b> (10th Schedule)                                                               |
| <b>61st</b>                       | 1989 | Voting age <b>21</b> → <b>18</b>                                                                        |
| <b>73rd / 74th</b>                | 1992 | Panchayati Raj / Municipalities                                                                         |
| <b>86th</b>                       | 2002 | Right to Education (Art. 21A); 11th Fundamental Duty                                                    |
| <b>101st</b>                      | 2016 | <b>Goods and Services Tax (GST)</b>                                                                     |

## Polity — likely exam questions

- **[5]** Explain Article 371A and why it is special to Nagaland. (*See Section 2.*)
- **[5]** Distinguish between Fundamental Rights and Directive Principles of State Policy.
- **[5]** What are the five types of writs? Explain any two.
- **[10]** Describe the emergency provisions of the Indian Constitution.
- **[10]** Explain the composition and powers of the Rajya Sabha vs the Lok Sabha.
- **[5]** Why does Part IX (Panchayati Raj) not apply to Nagaland, and what governs local administration there instead?

## Polity — MCQ traps (with answers)

### Ten quick MCQ-style checks — click for answers

1. Constitution came into **force** on? → **26 January 1950** 2. Words added by the 42nd Amendment to the Preamble? → **Socialist, Secular, Integrity** 3. "Heart and soul of the Constitution" (Ambedkar)? → **Article 32** (Right to Constitutional Remedies) 4. Right to Property is now under which article? → **Article 300A** (a legal right) 5. Which rights cannot be suspended in a National Emergency? → **Articles 20 and 21** 6. DPSP is borrowed from? → **Ireland** 7. Money Bill can be introduced only in? → **Lok Sabha** 8. Basic Structure Doctrine came from which case? → **Kesavananda Bharati (1973)** 9. Fundamental Duties were added on the recommendation of? → **Swaran Singh Committee** 10. GST was introduced by which

## SECTION 2 — ★ ARTICLE 371A: NAGALAND'S SPECIAL PROVISIONS

This is the single highest-yield item in the entire GK syllabus for you. It is Nagaland-specific, it is constitutional (so the facts are fixed and cannot be argued), and a Nagaland state examiner is very likely to set it. Learn every clause. It is equally usable as an objective answer, a 5-mark short answer, and a paragraph in almost any essay on the North-East.

### ● Core — virtual memory & I/O

Source: `_Shared/CORE_02_ComputerOrganization.md`

#### D5. Virtual memory

##### Concept

Two problems, one solution.

**Problem 1:** a program may be bigger than physical RAM. Historically programmers solved this by hand with "overlays" — manually swapping pieces in and out. Miserable.

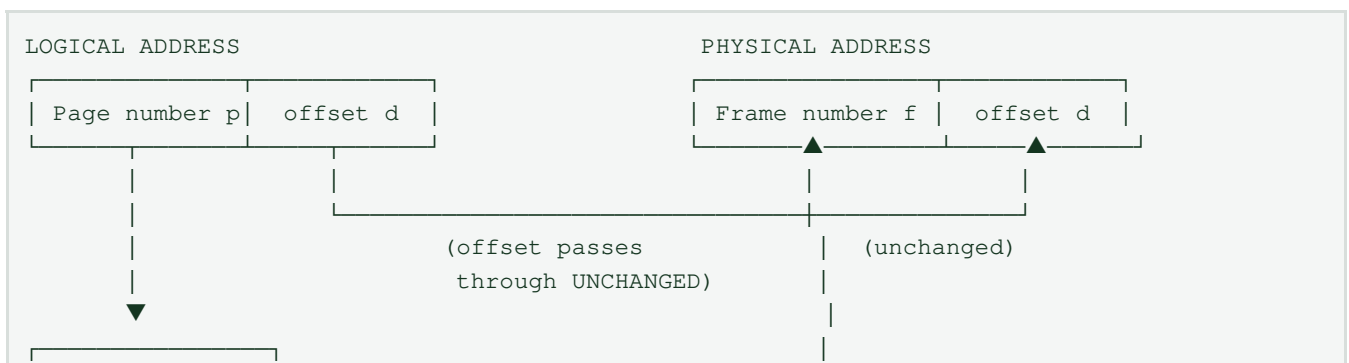
**Problem 2:** you want to run several programs at once, each believing it owns the whole address space, without them being able to corrupt each other.

**Virtual memory solves both.** Each process is given a large, private, contiguous **logical (virtual) address space**. The hardware (**MMU**) plus the OS translate every virtual address into a **physical address** at run time. Only the pages currently in use need be resident in RAM; the rest sit on disk and are fetched on demand. The programmer writes as if RAM were unlimited and private; the hardware maintains the illusion.

**The essential point to state in an exam:** virtual memory gives you (a) a logical address space larger than physical memory, (b) memory protection and isolation between processes, (c) relocation — a program can be loaded anywhere, and (d) efficient sharing of pages between processes. The cost is address-translation overhead and page-fault latency.

##### Paging

- The **logical address space** is divided into fixed-size **pages**.
- The **physical address space** is divided into equal-size **frames**.
- **Page size = frame size**, always, and always a **power of 2** (typically 4 KB).
- The **page table** maps page numbers to frame numbers, one per process.



|               |         |
|---------------|---------|
| PAGE TABLE    | entry p |
| (in memory,   |         |
| base in PTBR) |         |

### Address arithmetic — the rules:

If page size =  $2^n$  bytes, offset field =  $n$  bits  
 If logical address =  $m$  bits, page number field =  $(m - n)$  bits  
 Number of pages =  $2^{(m-n)}$   
 Number of frames = physical memory size / page size

### Worked example 13 — address field sizes.

A system has a 32-bit logical address, a 4 KB page size and 1 GB of physical memory.  
 Find the number of pages, the number of frames, the size of the offset, and the page table size if each entry is 4 bytes.

Page size = 4 KB =  $2^{12}$  → offset = 12 bits  
 Page number field =  $32 - 12 = 20$  bits  
 Number of pages =  $2^{20} = 1,048,576$  pages  
  
 Physical memory = 1 GB =  $2^{30}$  → physical address = 30 bits  
 Frame number field =  $30 - 12 = 18$  bits  
 Number of frames =  $2^{18} = 262,144$  frames  
  
 Page table size =  $2^{20}$  entries × 4 bytes = 4 MB PER PROCESS

**That 4 MB figure is the point of the question** — the page table is far too big to keep in registers and too big to want a copy per process in RAM. It motivates **multilevel page tables**, **inverted page tables** and, above all, the **TLB**.

### Worked example 14 — actual address translation.

Page size 4 KB. The page table shows page 3 → frame 10 (0x00A). Translate logical address 0x00003F7A.

Logical address 0x00003F7A = 0000 0000 0000 0000 0011 1111 0111 1010  
  
 Offset = low 12 bits = 1111 0111 1010 = 0xF7A  
 Page number = high 20 bits = 0x00003 = 3  
  
 Page 3 → Frame 0x00A (from the page table)  
  
 Physical address = frame number || offset  
                       = 0x00A || 0xF7A  
                       = 0x00AF7A  
  
 Sanity check in decimal: frame 10 × 4096 = 40960; offset 0xF7A = 3962  
                               40960 + 3962 = 44922 = 0xAF7A ✓

### Page table entry contents

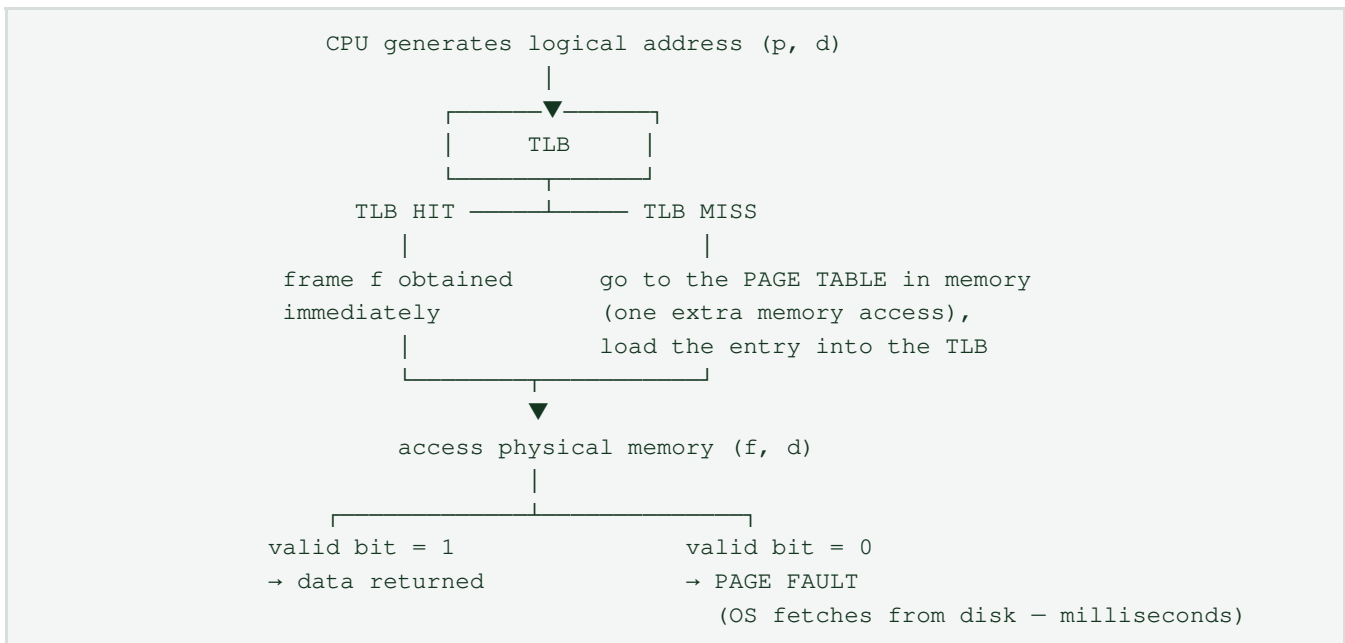
| Field                | Purpose                                                           |
|----------------------|-------------------------------------------------------------------|
| Frame number         | Where the page lives in physical memory                           |
| Valid / Present bit  | Is the page currently in RAM? If 0 → <b>page fault</b>            |
| Dirty / Modified bit | Has it been written? If not, no need to write it back on eviction |

|                          |                                                  |
|--------------------------|--------------------------------------------------|
| Reference / Accessed bit | Used by LRU-approximation replacement algorithms |
| Protection bits          | Read / Write / Execute permissions               |
| Caching disabled bit     | For memory-mapped I/O regions                    |

## The TLB (Translation Lookaside Buffer)

**The problem it solves:** without help, every single memory reference now takes **two** memory accesses — one to read the page table entry and one to get the actual data. That halves your memory performance.

**The solution:** the TLB is a **small, fully associative, very fast hardware cache of recently-used page-table entries** (typically 64–1024 entries), sitting inside the MMU. Thanks to locality, the hit rate is usually 98–99%.



### Effective Access Time with a TLB:

$$\begin{aligned}
 \text{EAT} &= h(t_{\text{TLB}} + t_{\text{mem}}) + (1 - h)(t_{\text{TLB}} + 2 \cdot t_{\text{mem}}) \\
 &= t_{\text{TLB}} + t_{\text{mem}} + (1 - h) \cdot t_{\text{mem}}
 \end{aligned}$$

(TLB hit: look in TLB, then one memory access for data.

TLB miss: look in TLB, one access for the page table, one for the data.)

### Worked example 15 — TLB effective access time.

*TLB access = 20 ns, main memory access = 100 ns, TLB hit ratio = 80%. Find the EAT.*

$$\begin{aligned}
 \text{TLB hit (80\%): } &20 + 100 = 120 \text{ ns} \\
 \text{TLB miss (20\%): } &20 + 100 + 100 = 220 \text{ ns}
 \end{aligned}$$

$$\begin{aligned}
 \text{EAT} &= 0.80 \times 120 + 0.20 \times 220 \\
 &= 96 + 44 \\
 &= 140 \text{ ns}
 \end{aligned}$$

Compare: with no paging at all it would be 100 ns. Slowdown = 40%.

If the hit ratio rises to 98%:

$$\text{EAT} = 0.98 \times 120 + 0.02 \times 220 = 117.6 + 4.4 = 122 \text{ ns} \rightarrow \text{slowdown only 22\%}.$$

(Some textbooks omit the TLB access time on a hit, giving  $\text{EAT} = 0.8 \times 100 + 0.2 \times 200 = 120 \text{ ns}$ . Always state which convention you are using, then you cannot be marked wrong.)

## Page faults

A **page fault** occurs when the valid bit is 0 — the page is not in physical memory. The sequence:

1. MMU raises a page-fault trap → control transfers to the OS
2. OS checks the reference is legal (else → segmentation fault, kill the process)
3. Find a free frame; if none, run the PAGE REPLACEMENT algorithm to evict one (write it back to disk first if the dirty bit is set)
4. Schedule a disk read of the required page into the frame
5. The process **BLOCKS**; the CPU runs something else meanwhile
6. On disk-interrupt: update the page table (valid bit = 1, frame number), update the TLB
7. **RESTART** the instruction that faulted

### Effective access time with page faults:

$$\text{EAT} = (1 - p) \times \text{memory\_access} + p \times \text{page\_fault\_service\_time}$$

**Worked example 16.** Memory access 100 ns, page fault service 10 ms = 10,000,000 ns.

Page fault rate  $p = 0.001$ .

$$\begin{aligned}\text{EAT} &= 0.999 \times 100 + 0.001 \times 10,000,000 \\ &= 99.9 + 10,000 \\ &= 10,099.9 \text{ ns} \approx 10.1 \mu\text{s}\end{aligned}$$

That is 101× SLOWER than raw memory, from a fault rate of only 1 in 1000.  
For less than 10% degradation you need  $p < 1$  in ~1,000,000.

**This calculation is the single best justification for why page-replacement algorithms matter** — quote it. (Page replacement algorithms themselves are worked in detail in **CORE\_03\_OperatingSystems.md §F.**)

## Segmentation and segmented paging

|                    | Paging                                                         | Segmentation                                                             |
|--------------------|----------------------------------------------------------------|--------------------------------------------------------------------------|
| Division unit      | <b>Fixed-size</b> pages                                        | <b>Variable-size</b> segments                                            |
| Chosen by          | The <b>hardware/OS</b> — invisible to the programmer           | The <b>programmer/compiler</b> — logical units (code, data, stack, heap) |
| Address form       | Page number + offset ( <b>one-dimensional</b> logical address) | Segment number + offset ( <b>two-dimensional</b> )                       |
| Fragmentation      | <b>Internal</b> (last page partly unused)                      | <b>External</b> (gaps between segments)                                  |
| Table entry        | Frame number                                                   | <b>Base + limit</b>                                                      |
| Protection/sharing | Per page — less natural                                        | <b>Per segment — natural</b> (a whole procedure or data structure)       |
| Compaction needed? | No                                                             | Sometimes                                                                |

**Segmented paging** combines both: divide the program into segments, then divide each segment into pages. You get the logical structure of segmentation with the fragmentation behaviour of paging. Used by x86.

**Segmentation address translation:** physical = base[s] + d, **but first check**  $d < \text{limit}[s]$ , else raise a trap. The limit check is what provides protection.

## Likely exam questions — Virtual memory

| Q                                                                                                                                                                                                                                          | Marks |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| What is virtual memory? State four benefits.                                                                                                                                                                                               | 5     |
| What is a TLB and why is it needed?                                                                                                                                                                                                        | 5     |
| Distinguish between paging and segmentation.                                                                                                                                                                                               | 5     |
| Explain address translation in a paged system with a diagram. A system has a 32-bit logical address and 4 KB pages; find the number of pages, the offset size and the page table size if each entry is 4 bytes.                            | 10    |
| A TLB has an access time of 20 ns, memory 100 ns and a hit ratio of 80%. Compute the effective access time. Recompute for 98% and comment.                                                                                                 | 10    |
| Explain virtual memory in detail: the motivation, paging, the page table and its entry fields, the TLB, the page fault handling sequence, and effective access time. Show with a numerical example why a low page-fault rate is essential. | 15    |

## MCQ traps — Virtual memory

| Trap                                             | Truth                                                                                                                 |
|--------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Virtual memory increases the speed of a computer | It increases the <b>apparent size</b> of memory. It generally <b>decreases</b> speed slightly (translation + faults). |
| Page size can be any value                       | Must be a <b>power of 2</b> so the address splits cleanly with no division.                                           |
| Page and frame sizes may differ                  | They are <b>always equal</b> .                                                                                        |
| The offset is translated too                     | The offset <b>passes through unchanged</b> . Only the page number is translated.                                      |
| TLB is a cache of data                           | It is a cache of <b>page table entries</b> (address translations).                                                    |
| TLB is direct mapped                             | It is typically <b>fully associative</b> (small enough to afford it).                                                 |
| Paging causes external fragmentation             | Paging causes <b>internal</b> fragmentation; <b>segmentation</b> causes external.                                     |
| Page fault = hardware error                      | It is a <b>normal, expected event</b> handled by the OS.                                                              |
| A page fault is handled by hardware              | The <b>trap</b> is raised by hardware; the <b>handling</b> is done by the <b>OS</b> .                                 |

---

## PART E — I/O ORGANISATION

### E1. I/O addressing: memory-mapped vs isolated

#### Concept

An I/O device controller has registers — a data register, a status register, a control register. The CPU has to read and write them. **The question is: do those registers live in the same address space as memory, or in a separate one?**

|  |                   |                                         |
|--|-------------------|-----------------------------------------|
|  | Memory-mapped I/O | Isolated (I/O-mapped / port-mapped) I/O |
|--|-------------------|-----------------------------------------|

|                              |                                                                                                        |                                                                         |
|------------------------------|--------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Address space                | I/O registers occupy <b>part of the normal memory address space</b>                                    | I/O has its <b>own separate address space</b>                           |
| Instructions used            | <b>Ordinary memory instructions</b> (LOAD, STORE, MOV)                                                 | <b>Special I/O instructions</b> (IN, OUT)                               |
| Control signals              | Only MEMR / MEMW                                                                                       | Needs extra <b>IOR</b> / <b>IOW</b> signals (an M/IO <sup>̄</sup> line) |
| Address lines used           | All of them                                                                                            | Usually <b>fewer</b> (e.g. 8 lines → 256 ports)                         |
| Memory available to programs | <b>Reduced</b> — I/O steals address space                                                              | <b>Full</b> memory space available                                      |
| Programming flexibility      | <b>High</b> — the full instruction set (arithmetic, logical, bit operations) works on device registers | Limited — only IN/OUT, usually via the accumulator                      |
| Decoding hardware            | More address lines to decode                                                                           | Simpler                                                                 |
| Used by                      | Motorola 68000, ARM, MIPS, PowerPC, most RISC                                                          | <b>Intel x86</b> (which supports both)                                  |

**Linear selection** (named in the CF syllabus): a simplified decoding technique where each device's chip-select is driven directly by **one dedicated address line** rather than by a decoder. Cheap, but only  $n$  devices for  $n$  address lines, and large parts of the address space become unusable/aliased.

## E3. Buses

### Synchronous vs asynchronous buses

|                              | Synchronous bus                                                               | Asynchronous bus                                                     |
|------------------------------|-------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Timing                       | Governed by a <b>common clock</b> ; every event happens at a fixed clock edge | <b>No clock</b> — events are sequenced by <b>handshaking signals</b> |
| Speed of devices             | All transfers take the same time — must be paced to the <b>slowest</b> device | Each transfer takes exactly as long as <b>that device</b> needs      |
| Signals                      | Clock, address, data, read/write                                              | Master-ready / Slave-ready (or REQ/ACK) handshake lines              |
| Complexity                   | <b>Simple</b> control logic                                                   | More complex (handshake circuitry)                                   |
| Bus length                   | Limited — <b>clock skew</b> over long distances                               | Can be longer                                                        |
| Mixing fast and slow devices | <b>Inefficient</b>                                                            | <b>Efficient</b>                                                     |
| Examples                     | Processor–memory buses, PCI                                                   | SCSI, older I/O buses                                                |

#### Asynchronous handshake — the four-phase (fully interlocked) protocol:

1. Master places the address and asserts MASTER-READY
2. Slave decodes, prepares the data, and asserts SLAVE-READY
3. Master reads the data and de-asserts MASTER-READY
4. Slave de-asserts SLAVE-READY

Each edge is a response to the previous one — hence "fully interlocked".

The transfer takes exactly as long as it needs. No clock is involved.

**Bus arbitration** (deciding who becomes bus master when several want it):

| Method                     | How                                                                                                            |
|----------------------------|----------------------------------------------------------------------------------------------------------------|
| Daisy chaining             | Grant signal propagates serially; nearest device wins. Simple, but fixed priority and a fault breaks the chain |
| Polling                    | The arbiter polls each device in turn using a count on the poll lines. Priority is programmable                |
| Independent request        | Each device has its own request and grant line into a central arbiter. <b>Fastest</b> , but the most wires     |
| Centralised vs distributed | One arbiter chip vs. devices resolving among themselves                                                        |

## Standard interface buses (both CF and Diploma syllabi mention these)

| Bus                        | Type                                                                       | Notes                                                                                                       |
|----------------------------|----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| ISA                        | Parallel, 8/16-bit, 8 MHz                                                  | Legacy PC expansion bus                                                                                     |
| PCI                        | Parallel, 32/64-bit, 33/66 MHz, ~133–533 MB/s                              | <b>Plug and play</b> : processor-independent; was the standard expansion bus                                |
| AGP                        | Accelerated Graphics Port                                                  | Dedicated <b>point-to-point</b> port for the graphics card, direct access to system RAM. Superseded by PCIe |
| PCI Express (PCIe)         | <b>Serial</b> , point-to-point <b>lanes</b> (×1, ×4, ×8, ×16), full duplex | The current standard for graphics, SSDs (NVMe), and expansion                                               |
| USB                        | Serial, tiered-star topology, <b>hot-pluggable</b> , up to 127 devices     | USB 1.1 = 12 Mbps, 2.0 = 480 Mbps, 3.0 = 5 Gbps, 3.1 = 10 Gbps, 4 = 40 Gbps. Also supplies power            |
| IDE / PATA                 | Parallel ATA, 40/80-pin ribbon, 2 drives per channel (master/slave)        | Legacy disk interface                                                                                       |
| SATA                       | <b>Serial</b> ATA, point-to-point, thin cable, hot-swap                    | SATA I 1.5 Gbps, II 3 Gbps, III 6 Gbps. Current disk standard                                               |
| SCSI / SAS                 | Parallel / Serial Attached SCSI                                            | Servers; supports many devices per bus, high performance                                                    |
| Serial port (RS-232)       | One bit at a time                                                          | 9-pin D-sub; legacy                                                                                         |
| Parallel port (Centronics) | 8 bits at a time                                                           | 25-pin; legacy printers                                                                                     |
| PS/2                       | 6-pin mini-DIN                                                             | Legacy keyboard (purple) and mouse (green)                                                                  |
| HDMI / DisplayPort         | Digital audio-video                                                        | Modern display interfaces                                                                                   |

**Serial vs parallel — why serial won.** At low speeds parallel is faster (8 bits at once).

But at high speeds, **skew** (the bits of a parallel word arriving at slightly different times) and **crosstalk** between adjacent wires become limiting. A single, well-shielded, differential serial line can be clocked *far* higher, and you can add lanes for bandwidth. Hence PATA→SATA, PCI→PCIe, parallel port→USB. **This is a good 5-mark answer.**

## Likely exam questions — I/O organisation

| Q                                                                                                                                                                                                                                                          | Marks |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| Distinguish between memory-mapped I/O and isolated I/O.                                                                                                                                                                                                    | 5     |
| What is DMA? Why is it used?                                                                                                                                                                                                                               | 5     |
| Explain the three DMA transfer modes.                                                                                                                                                                                                                      | 5     |
| List the methods of identifying an interrupting device and explain daisy chaining.                                                                                                                                                                         | 5     |
| Compare programmed I/O, interrupt-driven I/O and DMA under at least seven headings.                                                                                                                                                                        | 10    |
| Explain DMA data transfer with a block diagram. Describe the DMA controller registers and the complete transfer sequence including bus request and bus grant.                                                                                              | 10    |
| Compare synchronous and asynchronous buses. Explain the four-phase handshaking protocol with a timing description.                                                                                                                                         | 10    |
| Discuss I/O organisation in a computer system: I/O addressing techniques (memory-mapped, isolated, linear selection), the three data transfer methods with their relative merits, interrupt handling and priority resolution, and DMA including its modes. | 15    |

## MCQ traps — I/O organisation

| Trap                                           | Truth                                                                                                                                    |
|------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| In DMA, data passes through the CPU            | <b>No</b> — that is the entire point. Data goes <b>directly</b> between the device and memory.                                           |
| DMA needs no CPU involvement at all            | The CPU <b>initialises</b> the DMAC and <b>handles the completion interrupt</b> .                                                        |
| Cycle stealing interrupts the CPU              | It <b>does not interrupt</b> — it steals a bus cycle, delaying the CPU by one cycle.                                                     |
| Burst mode is slowest                          | <b>Burst is the fastest transfer</b> ; transparent DMA is the slowest but least intrusive.                                               |
| Memory-mapped I/O uses IN/OUT instructions     | Memory-mapped uses <b>ordinary LOAD/STORE</b> . IN/OUT belong to <b>isolated I/O</b> .                                                   |
| NMI can be disabled                            | <b>Non-Maskable</b> — by definition it cannot.                                                                                           |
| Programmed I/O is faster than interrupt-driven | It is the <b>slowest</b> and wastes the most CPU.                                                                                        |
| Interrupts are checked mid-instruction         | Checked <b>at the end of the instruction cycle</b> (except for page faults/exceptions, which occur mid-instruction and cause a restart). |
| AGP is a general expansion bus                 | AGP is <b>dedicated to graphics only</b> , point-to-point.                                                                               |
| USB supports 128 devices                       | <b>127</b> devices (address 0 is reserved).                                                                                              |

---

## PART F — PIPELINING

### F3. Pipeline hazards — the three classes

A **hazard** is any condition that prevents the next instruction from executing in its designated clock cycle.

## 1. Structural hazards (resource conflicts)

**Cause:** two instructions need the **same hardware resource** in the same cycle. Classic case: a single unified memory — instruction I1 in MEM wants memory while I4 in IF also wants memory.

**Solutions:**

- **Duplicate the resource** — separate instruction and data caches (a Harvard-style split L1). This is the standard fix.
- **Stall** (insert a pipeline bubble) — always works, always costs cycles.
- **Pipeline the resource** — e.g. a pipelined multiplier.

## 2. Data hazards (dependency conflicts)

**Cause:** an instruction needs a result that a previous instruction has not yet produced.

```
I1:  ADD  R1, R2, R3      ← R1 is written in WB (cycle 5)
I2:  SUB  R4, R1, R5      ← R1 is READ in ID (cycle 3) → R1 IS NOT READY YET

Cycle:    1      2      3      4      5
I1:       IF     ID     EX     MEM     WB      ← R1 written here
I2:              IF     ID*    EX     MEM      ← R1 needed here (cycle 3) ✗
                ↑ HAZARD
```

The three dependency types:

| Type | Name                                         | Pattern                      | Occurs in a simple in-order pipeline?       |
|------|----------------------------------------------|------------------------------|---------------------------------------------|
| RAW  | Read After Write — <i>true dependency</i>    | I2 reads what I1 writes      | <b>Yes — this is the common one</b>         |
| WAR  | Write After Read — <i>anti-dependency</i>    | I2 writes what I1 reads      | Only with out-of-order execution            |
| WAW  | Write After Write — <i>output dependency</i> | Both write the same register | Only with out-of-order / multi-cycle stages |

Solutions to data hazards:

| Solution                               | How it works                                                                                                                                                                                                                                 |
|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Operand forwarding / bypassing</b>  | <b>The main hardware fix.</b> Route the ALU result directly from the EX/MEM latch back to the ALU input of the next instruction, <i>before</i> it is written to the register file. Eliminates most RAW stalls with <b>zero cycle penalty</b> |
| <b>Pipeline interlock / stalling</b>   | Hardware detects the dependency and inserts <b>bubbles</b> until the data is ready. Always works, costs cycles                                                                                                                               |
| <b>Compiler instruction reordering</b> | The compiler moves independent instructions into the gap                                                                                                                                                                                     |
| <b>NOP insertion</b>                   | The compiler inserts no-operations — simple but wastes cycles                                                                                                                                                                                |
| <b>Register renaming</b>               | Eliminates WAR and WAW by using different physical registers                                                                                                                                                                                 |

**Forwarding cannot eliminate the LOAD-USE hazard.** If I1 is `LW R1, 0(R2)` the data only arrives at the end of MEM (cycle 4), but a dependent I2 needs it at the start of its EX (cycle 4). **One stall cycle is unavoidable**, then forwarding covers the rest. This is a

### 3. Control hazards (branch hazards)

**Cause:** the pipeline fetches instructions sequentially, but a **branch** may redirect execution. Until the branch outcome is known (typically at the end of EX), the instructions already fetched behind it may be wrong and must be **flushed**.

**Branch penalty** = the number of cycles lost = the number of instructions that must be flushed.

**Solutions:**

| Solution                    | How it works                                                                                                                                                                                                                                                                                |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Stall until resolved        | Simple, maximum penalty                                                                                                                                                                                                                                                                     |
| Branch prediction — static  | Always predict not-taken (just keep fetching); or "backward taken, forward not taken" (loops branch backward and are usually taken)                                                                                                                                                         |
| Branch prediction — dynamic | A <b>Branch History Table</b> / <b>Branch Target Buffer</b> records what each branch did last time. <b>2-bit saturating counters</b> are standard (a branch must mispredict twice to flip the prediction, so a loop's single exit doesn't poison it). Modern predictors exceed 95% accuracy |
| Delayed branch              | The ISA defines the instruction(s) <i>after</i> the branch as always executed (the <b>branch delay slot</b> ); the compiler fills the slot with useful work. Used in MIPS and SPARC                                                                                                         |
| Branch target buffer        | Caches the target address so the pipeline can start fetching immediately                                                                                                                                                                                                                    |
| Early branch resolution     | Move the comparison hardware into the ID stage, reducing the penalty from 3 cycles to 1                                                                                                                                                                                                     |
| Speculative execution       | Execute down the predicted path and discard the results if wrong                                                                                                                                                                                                                            |

### Worked example 19 — effective CPI with hazards

A 5-stage pipeline has an ideal CPI of 1. 20% of instructions are branches with a 3-cycle misprediction penalty and a 10% misprediction rate. 30% of instructions cause a load-use stall of 1 cycle. Find the effective CPI and the resulting speedup over a non-pipelined machine that takes 5 cycles per instruction.

```
Branch stall cycles per instruction = 0.20 × 0.10 × 3 = 0.06
Load-use stall cycles per instruction = 0.30 × 1           = 0.30
Total stalls per instruction           = 0.36
```

```
Effective CPI = 1 + 0.36 = 1.36
```

```
Speedup over non-pipelined = 5 / 1.36 = 3.68
(instead of the ideal 5 — hazards cost us 26% of the theoretical benefit)
```

## F4. Types of pipeline and related terms

| Term | Meaning |
|------|---------|
|      |         |

|                                |                                                                                                                                                                                 |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Instruction pipeline</b>    | Overlaps the phases of instruction execution (the 5-stage pipeline above)                                                                                                       |
| <b>Arithmetic pipeline</b>     | Splits a single arithmetic operation into stages (floating-point add: compare exponents → align mantissas → add → normalise). Used in vector processors                         |
| <b>Superscalar</b>             | <b>Multiple pipelines in parallel</b> — more than one instruction issued per cycle. CPI can be <b>less than 1</b>                                                               |
| <b>Superpipelined</b>          | A <b>deeper</b> pipeline (more, shorter stages) allowing a higher clock rate                                                                                                    |
| <b>VLIW</b>                    | Very Long Instruction Word — the <b>compiler</b> packs independent operations into one wide instruction; the hardware issues them in parallel with no dependency-checking logic |
| <b>Pipeline bubble / stall</b> | A cycle in which a stage does nothing, propagating down the pipeline                                                                                                            |
| <b>Flushing</b>                | Discarding wrongly-fetched instructions after a mispredicted branch                                                                                                             |

**Amdahl's Law** (worth one line): the overall speedup from improving one part of a system is limited by the fraction of time that part is used.

$$\text{Speedup}_{\text{overall}} = 1 / [ (1 - f) + f/s ]$$

where  $f$  is the fraction improved and  $s$  is the speedup of that fraction. If 60% of a program is parallelisable and you make that part infinitely fast, the maximum overall speedup is  $1/0.4 = 2.5\times$ . This is the fundamental limit on all parallel architecture.

## Likely exam questions — Pipelining

| Q                                                                                                                                                                                                                                                                                                   | Marks |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| What is pipelining? State the five stages of a typical instruction pipeline.                                                                                                                                                                                                                        | 5     |
| Define speedup, efficiency and throughput of a pipeline and give the formulas.                                                                                                                                                                                                                      | 5     |
| What is a data hazard? Explain RAW, WAR and WAW dependencies.                                                                                                                                                                                                                                       | 5     |
| A 5-stage pipeline has stage delays 5, 7, 4, 6 and 3 ns with 1 ns latches. Find the clock period, the speedup for a large $n$ , and the time for 1000 instructions.                                                                                                                                 | 10    |
| Explain the three classes of pipeline hazard and the techniques used to overcome each.                                                                                                                                                                                                              | 10    |
| Explain pipelining with a space-time diagram. Derive the speedup formula and show that the maximum speedup equals the number of stages. Discuss structural, data and control hazards with examples and their remedies, and explain why operand forwarding cannot fully eliminate a load-use hazard. | 15    |

## MCQ traps — Pipelining

| Trap                                                          | Truth                                                                                                             |
|---------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Pipelining reduces the execution time of a single instruction | It <b>does not</b> — it increases <b>throughput</b> . Latency per instruction stays the same or worsens slightly. |
| A $k$ -stage pipeline gives exactly $k\times$ speedup         | Only <b>asymptotically</b> and only with no hazards; the fill/drain overhead and stalls reduce it.                |
| Pipeline clock period = average stage delay                   | It equals the <b>slowest</b> stage delay + latch delay.                                                           |

|                                                 |                                                                                                          |
|-------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| RAW hazards can be fully removed by forwarding  | Not the <b>load-use</b> case — one stall is unavoidable.                                                 |
| WAR and WAW occur in a simple in-order pipeline | They appear with <b>out-of-order</b> execution.                                                          |
| Superscalar and superpipelined are the same     | <b>Superscalar</b> = multiple pipelines (issue width > 1). <b>Superpipelined</b> = more stages (deeper). |
| Branch prediction is done by the compiler only  | <b>Dynamic</b> prediction is done by <b>hardware</b> at run time.                                        |
| More stages is always better                    | Deeper pipelines have <b>larger branch penalties</b> and more latch overhead — there is an optimum.      |

---

---

## PART G — STORAGE AND HARDWARE (Diploma-focused)